Compiler Design

Final Exam — Short Answer Guide

NBIU | 18 Questions | Simple English

Q1. Define Finite Automata

A **Finite Automata (FA)** is an abstract machine used to recognize patterns. It has a finite number of states and transitions between them based on input symbols. It is used in lexical analysis of compilers.

Formal Definition — 5 Tuple: M = (Q, Σ, δ, q0, F)

Symbol	Meaning
Q	Finite set of states
Σ	Input alphabet (set of symbols)
δ	Transition function: δ(state, input) → next state
q0	Initial (start) state
F	Set of final (accepting) states

Two Types

Type	Key Property	Example use
DFA (Deterministic)	Exactly ONE next state per input	Used in real compilers
NFA (Non-deterministic)	Zero or more next states; allows ε-moves	Easier to design from regex

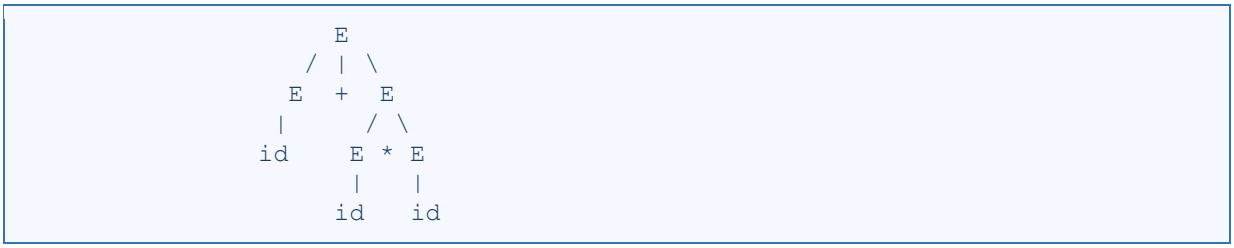
Q2. Illustrate the Parse Tree for a Given Grammar

A Parse Tree (Derivation Tree) shows how an input string is derived from a grammar, using a tree structure where root = start symbol, internal nodes = non-terminals, leaf nodes = terminals.

Example Grammar & Input String

```
Grammar:   E → E + E                    Input:  id + id * id
           E → E * E
           E → (E)
           E → id
```

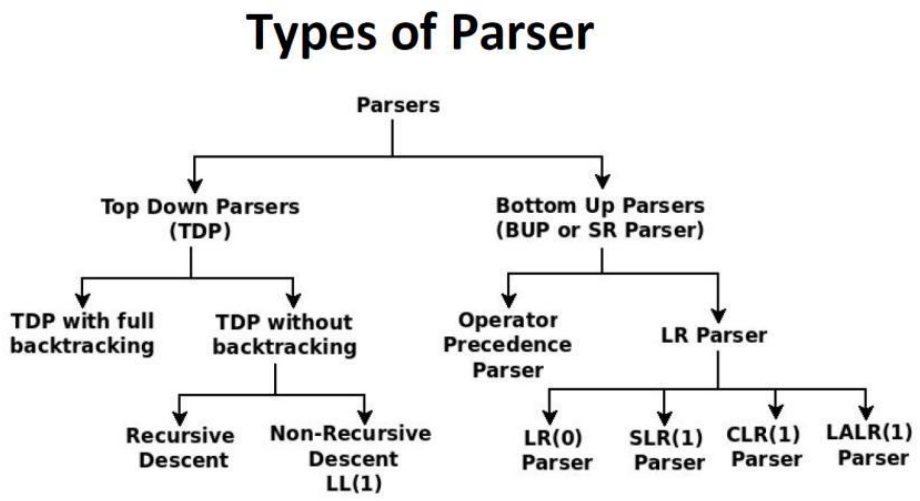
Parse Tree



Key Points

- Yield (read leaves left→right) = original input string: id + id * id
- If a string has 2 parse trees → grammar is AMBIGUOUS
- The grammar above IS ambiguous (operator precedence is undefined)

Q3. Different Types of Parser with Examples

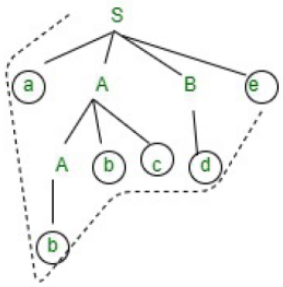


1. Top-Down Parsers :— start from Start Symbol, expand downward

String: abbcde

- **Example –**

S -> aABe
A -> Abc | b
B -> d



2. Bottom-Up Parsers :— start from input, reduce to Start Symbol

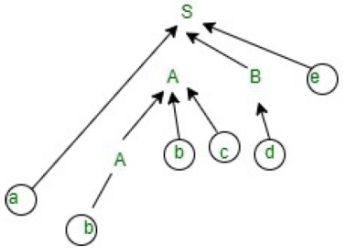
- Example –

S -> aABe ; A -> Abc | b ; B -> d

Now, let's consider the input to read and to construct a parse tree with bottom-up approach.

- First, you can start with A -> b.
- Now, expand A -> Abc.
- After that Expand B-> d.
- In the last, just expand the S -> aABe
- Final string, you will get **abbcde**.

Q: Write short notes on bottom-up parser.



Q4. Recursive-Descent Parser and Backtracking

Recursive-descent parser is a top-down parser that uses a set of recursive functions, one for each non-terminal. It tries to match the input by trying productions in order. If one production fails, it **backtracks** – returns to the previous choice point and tries the next alternative.

It means. If one derivation of a production fails, the syntax analyzer restarts the process using different rules of same production. This technique may process the input string more than once of determine the right production.

Structure

```
Grammar:  S → aAb  |  A → cd | c

procedure S() {  match('a');  A();  match('b');  }
procedure A() {
   if lookahead=='c' and next=='d': match('c'); match('d');
   else if lookahead=='c':          match('c');
   else: error();
}
```

Backtracking — when a choice fails, go back and try another

Grammar: S → aAb | aB A → cd | c B → cde Input: acde

```
Try S → aAb:
  match 'a' ✓ → try A → cd → match 'c','d' ✓ → expect 'b', got 'e' ✗ FAIL
  → BACKTRACK

Try S → aB:
  match 'a' ✓ → try B → cde → match 'c','d','e' ✓ → SUCCESS ✓
```

Avoiding Backtracking

- Use FIRST sets to pick the right production (no guessing)
- Remove left recursion → A → Aα becomes A → βA' A' → αA' | ε
- Apply left factoring to remove common prefixes

Q5. Construction procedure of LL(1) parsing table. (Mention the steps and rules)

The LL(1) table tells the parser: for non-terminal A on stack and terminal a in input, which production to use.

Pre-steps

1. Remove Left Recursion
2. Apply Left Factoring
3. Compute FIRST() sets
4. Compute FOLLOW() sets

Rules to Fill Table M[A, a]

Rule	Condition	Action
Rule 1	For each terminal a ∈ FIRST(α)	Add A → α to M[A, a]
Rule 2	If ε ∈ FIRST(α), for each b ∈ FOLLOW(A)	Add A → α to M[A, b]
Rule 3	If ε ∈ FIRST(α) and $ ∈ FOLLOW(A)	Add A → α to M[A, $]
Rule 4	All other cells	Mark as error

Example Table (Grammar: E→TE', E'→+TE'|ε, T→FT', T'→*FT'|ε, F→(E)|id)

NT	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

Note: If any cell has 2 productions → grammar is NOT LL(1) (conflict).

	id	(	)	*	+	$
E	E → TE′	E → TE′				
E′			E′ → ε		E′ → +TE′	E′ → ε
T	T → FT′	T → FT′				
T′			T′ → ε	T′ → *FT′	T′ → ε	T′ → ε
F	F → id	F → (E)				

Rules:

1. All the ε-productions are placed under FOLLOW sets.
2. Remaining productions are placed under the FIRSTs.

	FIRST	FOLLOW
E → TE′	{id,(}	{$,)}
E′ → +TE′ I ε	{+,ε}	{$,)}
T → FT′	{id,(}	{+,$,)}
T′ → *FT′ I ε	{*,ε}	{+,$,)}
F → id I (E)	{id,(}	{*,+,$,)}

Q6. FIRST() and FOLLOW() Sets and construct the LL(1) parsing table

FIRST(X) — terminals that can START a string derived from X

Case	Rule
X is a terminal	FIRST(X) = { X }
X → ε	Add ε to FIRST(X)
X → Y1 Y2...Yk	Add FIRST(Y1)−{ε}. If ε∈FIRST(Y1), also add FIRST(Y2)−{ε}, and so on.

FOLLOW(A) — terminals that can COME AFTER non-terminal A

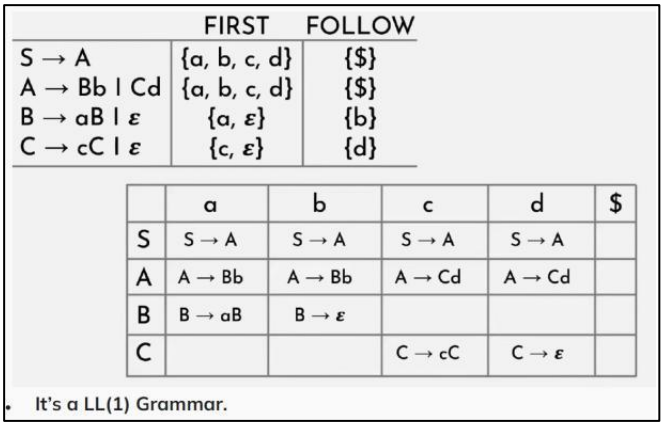
Case	Rule
A is Start Symbol	Add $ to FOLLOW(A)
B → αAβ (β is not empty)	Add FIRST(β)−{ε} to FOLLOW(A)
B → αA OR ε ∈ FIRST(β)	Add FOLLOW(B) to FOLLOW(A)

Worked Example

```
Grammar:  E → TE'
          E' → +TE'|ε
          T → FT'
          T' → *FT'|ε
          F → (E)|id
```

LL(1) parsing table

Non-Terminal	FIRST Set	FOLLOW Set
E	{ id, (}	{ $,) }
E'	{ +, ε }	{ $,) }
T	{ id, (}	{ +, $,) }
T'	{ *, ε }	{ +, $,) }
F	{ id, (}	{ *, +, $,) }



	FIRST	FOLLOW
S → A	{a, b, c, d}	{$}
A → Bb | Cd	{a, b, c, d}	{$}
B → aB | ε	{a, ε}	{b}
C → cC | ε	{c, ε}	{d}

	a	b	c	d	$
S	S → A	S → A	S → A	S → A	
A	A → Bb	A → Bb	A → Cd	A → Cd	
B	B → aB	B → ε			
C			C → cC	C → ε	

- It's a LL(1) Grammar.

Q7. Three-Address Code (TAC)

Three-Address Code (TAC) is an intermediate code used in compilers where each instruction contains at most three addresses. It breaks complex expressions into simple statements of the form x = y op z. TAC improves code optimization and makes machine code generation easier. It is commonly represented using quadruples, triples, and indirect triples.

General Form

a=b **op** c *a=b **op** c*

- a, b, c are operands (constants, names, temporaries).
- op is an operator (+, -, *, etc.)

Given Expression: *a = b + c * d*

TAC: **t1 = c * d**

t2 = b + t1

a = t2

Another Expression: ***x = (a + b) * (c - d)***

TAC: ***t1 = a + b***

t2 = c - d

t3 = t1 * t2

x = t3

Example: a = b * (-c) + b * (-c)

```
t1 = -c
t2 = b * t1
t3 = -c
t4 = b * t3
t5 = t2 + t4
a  = t5
```

Advantages: Easy to translate to assembly, easy to optimize.
Disadvantage: Generates many temporary variables.

Q8. Quadruple, Triples and Indirect Triples

1. Quadruple — 4 fields: (op, arg1, arg2, result)

Question – Write quadruple, triples and indirect triples for following expression : (x + y) * (y + z) + (x + y + z)

Explanation – The three address code is:

t1 = x + y
t2 = y + z
t3 = t1 * t2
t4 = t1 + z
t5 = t3 + t4

#	Op	Arg1	Arg2	Result
(1)	+	x	y	t1
(2)	+	y	z	t2
(3)	*	t1	t2	t3
(4)	+	t1	z	t4
(5)	+	t3	t4	t5

Quadruple representation

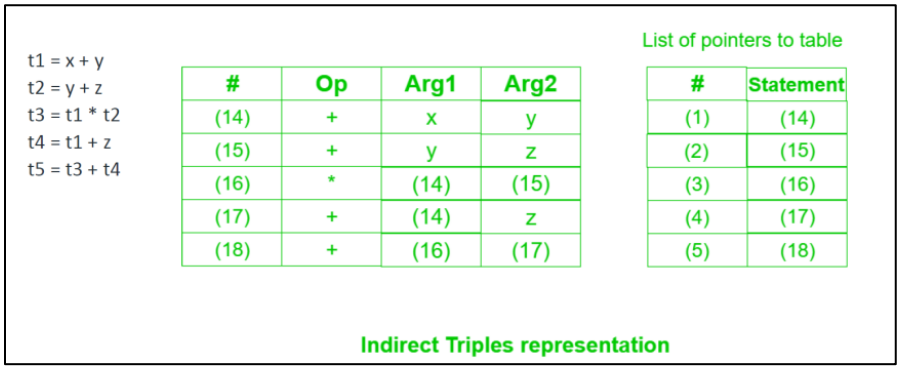
2. Triples — 3 fields: (op, arg1, arg2) — result referred by index number

t1 = x + y
t2 = y + z
t3 = t1 * t2
t4 = t1 + z
t5 = t3 + t4

#	Op	Arg1	Arg2
(1)	+	x	y
(2)	+	y	z
(3)	*	(1)	(2)
(4)	+	(1)	z
(5)	+	(3)	(4)

Triples representation

3. Indirect Triples — same Triples table + a pointer array (IP)



#	Op	Arg1	Arg2
(14)	+	x	y
(15)	+	y	z
(16)	*	(14)	(15)
(17)	+	(14)	z
(18)	+	(16)	(17)

#	Statement
(1)	(14)
(2)	(15)
(3)	(16)
(4)	(17)
(5)	(18)

Triples table stays the same. Reordering = just change the IP array.

Q9. Comparison: Quadruple vs Triples vs Indirect Triples

Feature	Quadruple	Triples	Indirect Triples
Fields	4 (op,arg1,arg2,result)	3 (op,arg1,arg2)	Triple + pointer table
Result stored	Explicitly (temp var)	By position index	By position via pointer
Memory	Higher	Lower	Slightly > Triples
Optimization / Reorder	Easy	Hard (refs break)	Easy (reorder pointer)
Common Sub-expr Elim.	Easy	Harder	Easy
Complexity	Simple	Simple	Slightly complex
Used in	Most real compilers	Simple compilers	Optimizing compilers

Advantages / Disadvantages Summary

- Quadruple ☑ Easy to implement, easy reordering ✗ More memory
- Triples ☑ Less memory ✗ Hard to reorder (optimization difficult)
- Indirect Triples ☑ Less memory + easy reorder ✗ Extra pointer array needed

Q10. Symbol Table and Its Importance

A Symbol Table is a data structure maintained by the compiler to store information about all identifiers (variables, functions, classes) used in the source program.

Importance / Uses

- To store names of all entities in a structured form at one place
- To verify if a variable has been declared
- To implement type checking, by verifying assignments and expressions in the source code are semantically correct.
- Its used by the compiler to achieve compile-time efficiency

Example Table:

Name	Type	Size	Line declared	Address
x	int	4	5	100
arr	char	10	6	104

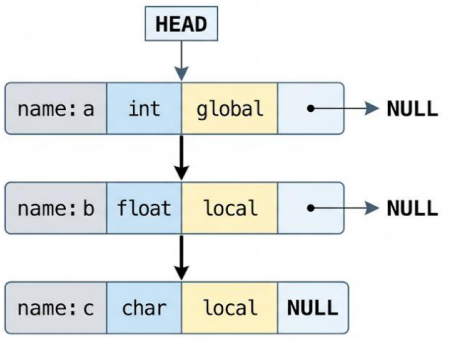
Q11. Linked List and Hash Table for Symbol Table Implementation

A symbol table is a data structure used in compiler design to store information about identifiers such as name, type, scope, and memory location. It can be implemented using different techniques. Two common methods are Linked List and Hash Table.

1. Linked List implementation

In this method, each symbol is stored as a **node in a linked list**. Each node contains information about the identifier and a pointer to the next node.

Diagram:



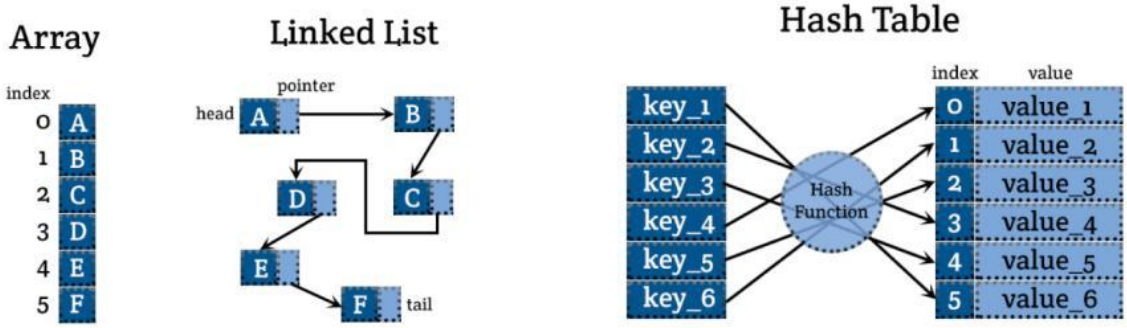
2. Hash Table implementation

In this method, a **hash function** is used to compute an index where the identifier is stored in a table. Each position in the table is called a bucket.

Diagram:

Index
0 → NULL
1 → a → c → d
2 → NULL
3 → b
4 → NULL

Linked list implementation of a symbol table is simple but inefficient due to linear searching, while hash table implementation is more efficient as it provides fast average-case access using hashing and collision handling. Therefore, hash tables are preferred in modern compiler design.



Q12. Division Hashing Method

Division Method:
This is the most simple and easiest method to generate a hash value. The hash function divides the value k by M and then uses the remainder obtained.
Formula:
h(K) = k mod M
Here,
k is the key value, and
M is the size of the hash table.
It is best suited that M is a prime number as that can make sure the keys are more uniformly distributed. The hash function is dependent upon the remainder of a division.

Hash Function

```
h(k) = k mod m    where k = key value,  m = hash table size
(use a prime number)
```

Example — keys: 25, 36, 14, 7, 52, 33 with m = 10

Key (k)	h(k) = k mod 10	Stored at Index
25	25 mod 10	5
36	36 mod 10	6
14	14 mod 10	4
7	7 mod 10	7
52	52 mod 10	2
33	33 mod 10	3

```
Hash Table:
Index | Key
------|-----
  0   |  -
  2   |  52
  3   |  33
  4   |  14
  5   |  25
  6   |  36
  7   |  7
```

Q13. Code Generation

Code Generation is the final phase of compiler design in which intermediate code is translated into target machine code or assembly code. It takes input such as three-address code or syntax tree and produces efficient machine instructions. It involves tasks like instruction selection, register allocation, and memory management. For example, the expression a =

b + c * d is first converted into intermediate code and then into assembly instructions using registers. Code generation ensures that the program is ready for execution on the target machine.

Types of Code Generation

1. Template-Based Code Generation
2. Model-Driven Code Generation
3. Compiler-Based Code Generation
4. Meta-Programming

Expression: ***x = (a + b) * (c - d)***

TAC: t1 = a + b
t2 = c - d
t3 = t1 * t2
x = t3

Machine Code:

```
MOV R1, a
ADD R1, b ;       R1 = a + b

MOV R2, c
SUB R2, d ;       R2 = c - d
MUL R1, R2 ;      R1 = (a+b)*(c-d)

MOV x, R1
```

Phases Involved in Code Generation:

- Instruction Selection
- Register Allocation
- Instruction Ordering
- Memory Management

Code Generation is a crucial phase of compiler design where intermediate code is converted into target machine code. It ensures that the source program becomes executable on the target machine while maintaining correctness and efficiency.

Q14. Code Generation Algorithm

The **Code Generation Algorithm** converts **Three Address Code (TAC)** into **target machine code (assembly code)** using registers and simple instruction rules.

Algorithm:

1. Start
2. Read Three Address Code
3. For each statement do:
    - If statement is x = y → generate MOV x, y
    - If statement is x = y op z →
        - □ Load y into register

- □ Apply operation with z
- □ Store result in x

4. Use registers efficiently
5. Repeat until all statements processed
6. End

The code generation algorithm is used to convert intermediate code into target machine code. Each three-address statement is processed one by one. For assignment statements, a MOV instruction is generated. For arithmetic expressions, values are loaded into registers, operations are performed, and results are stored back in memory. Efficient register usage is important for better performance. This algorithm helps in producing executable code from intermediate representation.

Q15. Code Generation Issues

Code generation is the final phase of compiler design where intermediate code is converted into machine code. During this phase, the compiler faces several **important problems (issues)** that affect efficiency and correctness.

1. **Input to code generator** – Intermediate representation (TAC, postfix, syntax tree) plus symbol table info. Must be error-free.
2. **Target program** – Output can be absolute machine language (immediate execution), relocatable (easy linking), or assembly (easier generation).
3. **Memory management** – Mapping names to addresses. Static allocation (fixed at compile time) vs stack allocation (activation records).
4. **Instruction selection** – Choose appropriate machine instructions for speed and size. Need complete and uniform instruction set.
5. **Register allocation** – Which variables to keep in registers? Register assignment – which register to use? Important for performance.
6. **Evaluation order** – Different orders may need fewer registers. Example: (a+b)+(c+d) vs ((a+b)+c)+d.

Diagram:
`[Input: IR + Symbol Table] --> Code Generator --> [Target code]`
`Issues: Memory, Registers, Instructions, Order`

Code generation in compiler design faces several issues. The main issues are instruction selection, register allocation, order of evaluation, memory management, instruction cost, machine dependency, and control flow handling. Instruction selection deals with choosing efficient machine instructions. Register allocation is difficult due to limited CPU registers. Order of evaluation ensures correct execution sequence. Memory management handles storage of variables and temporaries. These issues make code generation a complex but important phase in compiler design.

Q16. Code Optimization and Its Importance

Code optimization is a program transformation technique that improves intermediate code to consume fewer resources (CPU, memory) and produce faster machine code.

Importance:

1. **Cleaner code base** – removes redundant operations.
2. **Higher consistency** – uniform optimizations.
3. **Better readability** (for human after optimization?). Actually optimization makes code efficient.
4. **More efficient** – faster execution.
5. **Easier debugging** (optimized code may be harder, but proper optimization reduces bugs).
6. **Improved workflow** – less resource use.
7. **Easier maintenance** – smaller technical debt.
8. **Smaller code size** – important for embedded systems.

Optimization types:

- **Machine independent:** Loop optimization (code motion, fusion, unrolling), constant propagation, constant folding.
- **Machine dependent:** Peephole optimization, instruction-level parallelism.
  Example – Constant folding:
  Before: $x=2*3$ $x=2*3$
  After: $x=6$ $x=6$

Importance summary: Reduces execution time and memory usage.

Important: Optimization NEVER changes what the program does — only HOW efficiently it runs.

Q17. Control Flow Graph (CFG) for Given Code

A **Control Flow Graph (CFG)** represents the flow of control among basic blocks. Nodes are basic blocks, edges represent jumps/falls.

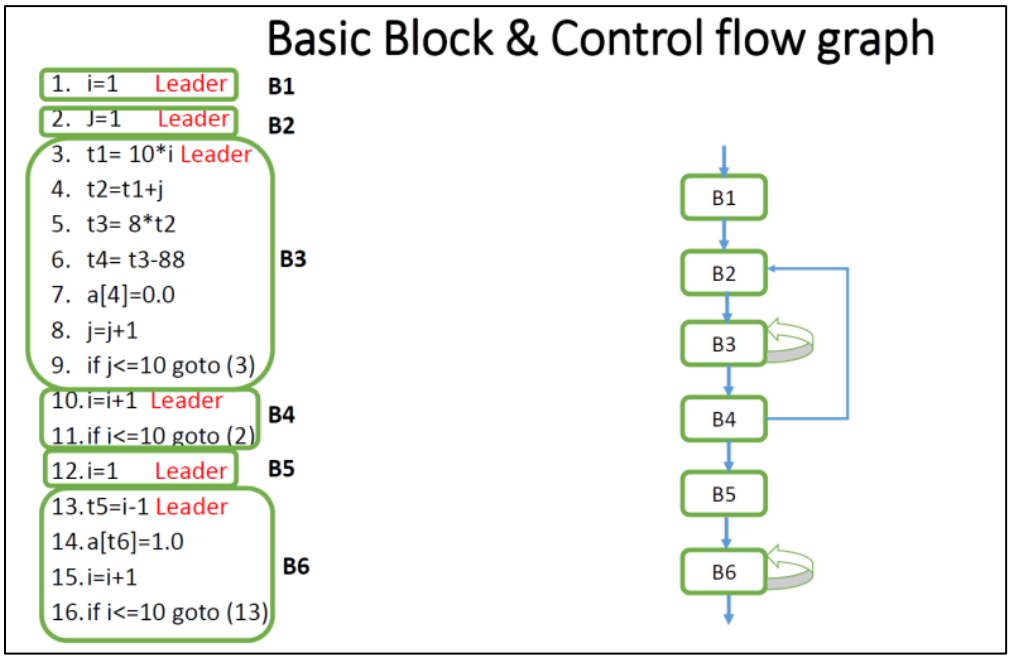
Given code (from PDF, page 16):

```
1. i=1          (leader)
2. J=1          (leader)
3. t1=5*i
4. t2=t1+J
5. t3=4*t2
6. t4=t3
7. a[t4]=-1
8. J=J+1
9. if J<=5 goto 3
10. i=i+1       (leader)
11. if i<5 goto 2
```

Basic blocks (leaders: 1,2,3,10):

- B1: statements 1
- B2: statements 2
- B3: statements 3-9 (from 3 to 9, because 9 jumps to 3, and 10 is next leader after 9)
- B4: statements 10-11 (10,11)

Example Code & CFG:



Q18. Compile-Time Errors vs Run-Time Errors

Aspect	Compile-time errors	Run-time errors
When detected	During compilation	During program execution
Prevention	Program cannot run until fixed	Program may crash or misbehave
Examples	Syntax error, type mismatch, undeclared variable	Division by zero, null pointer, array index out of bounds
Detection	Compiler reports with line number	Usually causes exception or crash
Cost	Cheap to fix	Costly (may happen in production)

Compile-Time Error Examples

Type	Example Code	Error Message
Syntax Error	int x = ;	expected expression
Type Error	int x = "hello";	incompatible types
Undeclared Var	y = 5; (y not declared)	'y' undeclared
Missing semicolon	x = 5	expected ';'

Run-Time Error Examples

Type	Example Code	What happens
Division by zero	int x = 5/0;	Program crashes
Null pointer deref	int *p=NULL; *p=5;	Segmentation fault
Array out of bounds	int a[5]; a[10]=1;	Seg fault / garbage
Stack overflow	Infinite recursion	Program crashes
Infinite loop	while(true) { }	Program hangs

Code Example Comparison

```
// COMPILE-TIME ERROR — caught before running:
int main() {
   int x = "hello";   // ← type error
   printf("%d" x);    // ← syntax error (missing comma)
}

// RUN-TIME ERROR — compiles fine but crashes:
int main() {
   int arr[5];
   arr[10] = 99;       // ← out-of-bounds at runtime
   int y = 10 / 0;     // ← division by zero at runtime
}
```